

AI ASSISTED CODING

ASSIGNMENT – 8.1

NAME: Geethanjali

HTNO: 2303A51005

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

- Task: Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

- Requirements:

- o Password must have at least 8 characters. o

Must include uppercase, lowercase, digit, and special character.

- o Must not contain spaces. Example Assert

Test Cases:

```
assert is_strong_password("Abcd@123") == True
```

```
assert is_strong_password("abcd123") == False assert
```

```
is_strong_password("ABCD@1234") == True
```

Expected Output #1:

- Password validation logic passing all AI-generated test cases.

The screenshot shows a code editor window with a file named testcasedemo.py. The script defines a function is_Strong_password(password) which checks if a password is at least 8 characters long and contains at least one character from each category: uppercase letters, lowercase letters, digits, and special characters. It also includes three assertion statements: assert is_Strong_password("Abcdefl!") == True, assert is_Strong_password("abcdefl!") == True, and assert is_Strong_password("ABCDEFGH1!") == False. Below the code, the TERMINAL tab displays the command prompt output. The first two assertions pass, but the third fails, resulting in an AssertionError message. The user's cursor is positioned at the end of the command prompt line.

```
Users > geethanjali > Downloads > testcasedemo.py > ...  
  
1 def is_Strong_password(password):  
2     if len(password)<8:  
3         return False  
4     has_upper=any(c.isupper() for c in password)  
5     has_lower=any(c.islower() for c in password)  
6     has_digit=any(c.isdigit() for c in password)  
7     has_special=any(c in "!@#$%^&*()-_+=[]{}|;:,.<>?/" for c in password)  
8     return has_upper and has_lower and has_digit and has_special  
9 assert is_Strong_password("Abcdefl!") == True  
10 assert is_Strong_password("abcdefl!") == True  
11 assert is_Strong_password("ABCDEFGH1!") == False  
12  
13
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

/usr/local/bin/python3 /Users/geethanjali/Downloads/testcasedemo.py
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/testcasedemo.py
Traceback (most recent call last):
File "/Users/geethanjali/Downloads/testcasedemo.py", line 10, in <module>
 assert is_Strong_password("abcdefl!") == True
AssertionError
geethanjali@Geethanjali / % █

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

- Task: Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

- Requirements:

- o Classify numbers as Positive, Negative, or Zero.

- o Handle invalid inputs like strings and None.

Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

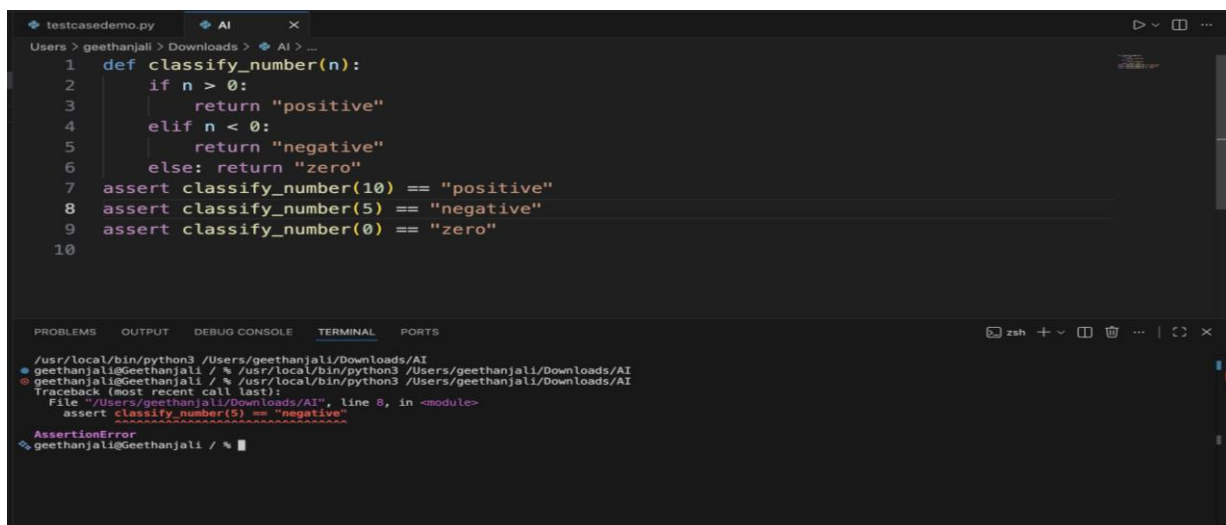
```
assert classify_number(10) == "Positive" assert
```

```
classify_number(-5) == "Negative" assert
```

```
classify_number(0) == "Zero"
```

Expected Output #2:

- Classification logic passing all assert tests.



The screenshot shows a code editor with a file named `testcasedemo.py`. The code defines a function `classify_number(n)` that returns "positive", "negative", or "zero" based on the input `n`. Below the function, there are three assert statements: `assert classify_number(10) == "positive"`, `assert classify_number(5) == "negative"`, and `assert classify_number(0) == "zero"`. The terminal output shows the execution of the script, which results in an `AssertionError` because the second assert statement is failing: `assert classify_number(5) == "negative"`.

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

- Task: Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

- Requirements:

- o Ignore case, spaces, and punctuation.

- o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

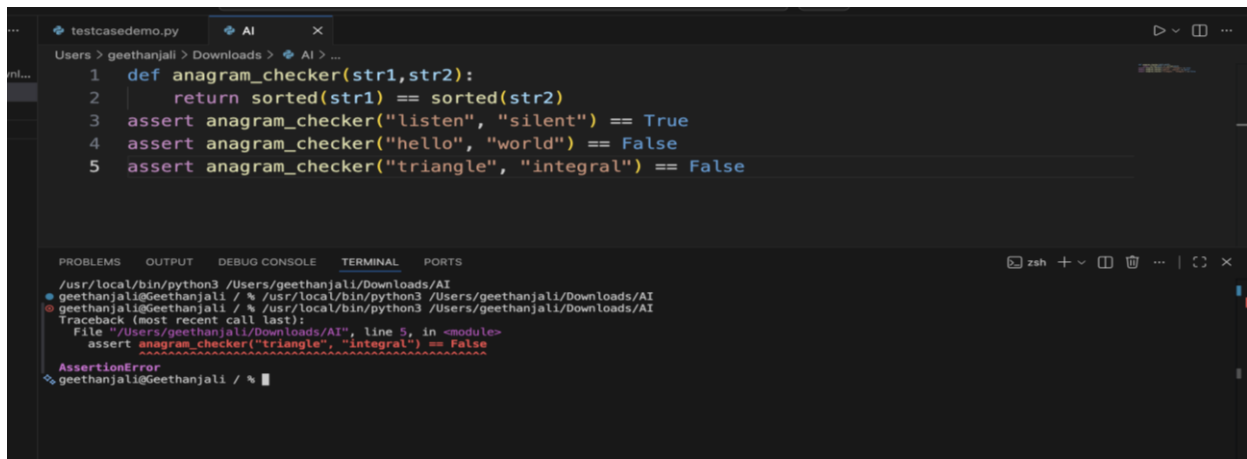
`assert is_anagram("listen", "silent") == True` assert

`is_anagram("hello", "world") == False` assert

`is_anagram("Dormitory", "Dirty Room") == True`

Expected Output #3:

- Function correctly identifying anagrams and passing all AI- generated tests.



The screenshot shows a code editor with a file named `testcasedemo.py`. The code defines a function `anagram_checker(str1, str2)` that returns `sorted(str1) == sorted(str2)`. Below the function, there are three assert statements: `assert anagram_checker("listen", "silent") == True`, `assert anagram_checker("hello", "world") == False`, and `assert anagram_checker("triangle", "integral") == False`. The terminal output shows the execution of the script, which fails on the third assert statement, resulting in an `AssertionError`.

```
1 def anagram_checker(str1, str2):
2     return sorted(str1) == sorted(str2)
3     assert anagram_checker("listen", "silent") == True
4     assert anagram_checker("hello", "world") == False
5     assert anagram_checker("triangle", "integral") == False

Traceback (most recent call last):
  File "/Users/geethanjali/Downloads/AI", line 5, in <module>
    assert anagram_checker("triangle", "integral") == False
AssertionError
```

Task Description #4 (Inventory Class – Apply AI to Simulate Real- World Inventory System)

- Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

• Methods:

`add_item(name, quantity)`

`remove_item(name, quantity)`

`get_stock(name)`

Example Assert

Test Cases: `inv = Inventory()`

`inv.add_item("Pen", 10)` assert

`inv.get_stock("Pen") == 10`

`inv.remove_item("Pen", 5)` assert

`inv.get_stock("Pen") == 5`

`inv.add_item("Book", 3)` assert

`inv.get_stock("Book") == 3`

Expected Output #4:

- Fully functional class passing all assertions.

```

85 class Inventory:
86     def __init__(self, name, quantity):
87         self.name = name
88         self.quantity = quantity
89     def add_item(self, name, quantity):
90         self.name = name
91         self.quantity += quantity
92     def remove_item(self, name, quantity):
93         if self.name == name and self.quantity >= quantity:
94             self.quantity -= quantity
95         else:
96             return "Not enough items in inventory"
97     def get_stock(self, name):
98         if self.name == name: return self.quantity
99 inv = Inventory()
100 inv.add_item("Pen", 10)
101 assert inv.get_stock("Pen") == 10
102 inv.remove_item("Pen", 5)
103 assert inv.get_stock("Pen") == 5
104 inv.add_item("Book", 3)
105 assert inv.get_stock("Book") == 3

```

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

- Requirements:

o Validate "MM/DD/YYYY" format. o

Handle invalid dates. o Convert valid dates

to "YYYY-MM-DD".

Example Assert Test Cases:

`assert validate_and_format_date("10/15/2023") == "2023-10-15"` assert

`validate_and_format_date("02/30/2023") == "Invalid Date"` assert

`validate_and_format_date("01/01/2024") == "2024-01-01"` Expected

Output #5:

- Function passes all AI-generated assertions and handles edge cases.

```

107 def validate_and_format_date(date_str):
108     import re
109     pattern = r'^\d{2}/\d{2}/\d{4}$'
110     if not re.match(pattern, date_str):
111         return "Invalid date format"
112     month, day, year = map(int, date_str.split('/'))
113
114     # Days in each month
115     days_in_month = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
116
117     # Check for leap year
118     if year % 4 == 0 and (year % 100 != 0 or year % 400 == 0):
119         days_in_month[1] = 29
120
121     if month < 1 or month > 12 or day < 1 or day > days_in_month[month - 1]:
122         return "Invalid date"
123     return f"{year:04d}-{month:02d}-{day:02d}"
124 assert validate_and_format_date("10/15/2023") == "2023-10-15"
125 assert validate_and_format_date("02/30/2023") == "Invalid date"
126 assert validate_and_format_date("01/01/2024") == "2024-01-01"
127

```

PROBLEMS	OUTPUT	TERMINAL	PORTS
----------	--------	----------	-------

> ✓ TERMINAL

● PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/testcasedemo.py

○ PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>